

🌟 PROJECT: AI Eye Screening System (ENVISIONEERS)

🌀 1. Starting Point — Goal of the Project

We wanted to build a system that can:

- 👉 Use a camera
- 👉 Run AI models
- 👉 Detect eye diseases (Glaucoma, Cataract, DR)
- 👉 Show results in real time
- 👉 Save results for later

🌀 2. Deploying the Model (First Step)

We started with **pre-trained AI models**.

What we did:

- Got model files (from Edge AI / training output)
- These included:
 - model.onnx → the AI model
 - artifacts/ → compiled files for the TI hardware
 - config.yaml → tells system how to use the model

Then we placed them in:

/opt/models/

3. Connecting Model to the System

We used:

`/opt/edgeai-gst-apps/`

This is the **main application framework**.

Key files:

- `app_edgeai.py` → runs the AI pipeline
- `configs/*.yaml` → tells which model to run
- `post_process.py` → draws results on HDMI

4. Running the Model

We tested with:

`python3 app_edgeai.py configs/image_classification.yaml`

What happened:

- Camera started
- Frames were captured
- AI predicted results
- Output displayed on HDMI

5. Adding a Menu System

Instead of running commands manually, we built:

`model_menu.py`

This allowed:

[1] Glaucoma

[2] Cataract

[3] DR

👉 Now user can select model easily

6. Capturing Predictions

We modified the system to save predictions:

```
/tmp/last_prediction.txt
```

Format:

```
glaucoma|92.3
```

👉 This lets us use results later (summary, UI, etc.)

7. Saving Image Evidence

We added:

```
cv2.imwrite("/tmp/last_frame.jpg", frame)
```

What this does:

- saves the latest camera image
- used as **proof of diagnosis**

8. Creating a Summary File

After each run, we generate a report:

```
/opt/edgeai-gst-apps/runs/*.txt
```

Example:

Patient Name : John Doe

Model Name : Glaucoma Detection

Prediction : glaucoma

Confidence : 92.3%

Verdict : Disease Detected

Advice : Please see a doctor

9. Adding Patient Info

We added input:

Patient Name:

Patient ID:

👉 This made the system feel like a **real medical device**

10. Saving Image Evidence (Safe Version)

After scan ends:

Save image evidence? [y/n]

If yes:

- copy /tmp/last_frame.jpg
- store in:

/opt/edgeai-gst-apps/runs/

11. Viewing Past Results

We added:

[4] View Previous Results

👉 lets user open old summaries

12. Patient History System

We added:

[5] View Patient History

👉 search by name:

Enter patient name: john_doe

👉 shows all past scans

13. HDMI Result Screen (BIG FEATURE)

After scan ends:

We generate a **visual card**:

SCAN COMPLETE

Prediction: Cataract

Confidence: 91%

Then display it on HDMI using:

gst-launch-1.0 ... kmssink

14. Showing Captured Image on HDMI

After result card:

👉 we show the actual image taken from camera

Flow:

Result Card → Camera Image

👉 This makes it feel like a **real diagnostic system**

15. Branding (Final Polish)

We changed:

ENVISIONOR → ENVISIONEERS

Added:

ENVISIONEERS LAB

AI EYE SCREENING SYSTEM

16. Fixing Time Issue

Problem:

1970-01-01

Solution:

```
date -s "2026-04-20 18:45:00"  
timedatectl set-timezone America/Chicago
```

👉 Now summaries show real timestamps

17. Final System Flow

When user runs system:

1. Select model
2. Enter patient info
3. Camera starts
4. AI detects disease
5. User stops scan
6. Save image? (optional)
7. Summary is generated
8. HDMI shows result card
9. HDMI shows captured image
10. Return to menu

FINAL RESULT (What You Built)

You built:

AI-powered Embedded Eye Screening System

With:

- real-time AI inference
- multiple disease detection
- HDMI visualization
- patient tracking
- image evidence saving
- history retrieval

Why this is impressive

This is not just AI.

This is:

- 👉 Embedded systems
- 👉 Computer vision
- 👉 UI/UX
- 👉 Data logging
- 👉 Medical-style workflow

If you explain this in interview

Say:

“We designed an embedded AI eye screening system that performs real-time disease detection, logs patient results, saves diagnostic evidence, and presents results through an HDMI-based interface.”